C Programming

Assignment 2 (CO2) - Report

Operators, Decision-Making Statements and Looping Constructs

Questions 1 to 4: Precedence and Associativity, Short-Circuit Evaluation, Bitwise Operators, else-if Ladder vs switch

Student name	P.SAI SUSHANTH REDDY
Register number	1925260171
Course / Faculty	CSA0221
GitHub repository	https://github.com/nagagowthamjanjanam-code/CSA02-1965260183-Assignment-2
Compiler used for measurements	GCC 13.3.0 (Ubuntu 24.04). Compile command: `gcc -Wall -Wextra -o qN qN.c`
Screenshots	Student's own execution screenshots from an online C compiler (Figures 1 to 5)

Fill in the name, register number, faculty and the GitHub link before submission. The assignment states that a report without a working GitHub link is not evaluated.

Contents and marks

Section	Topic	Marks
Q1	Precedence, associativity and the discount bug (K4)	6
Q2	When does short-circuit evaluation matter? (K4)	4
Q3	Device-status register with bitwise operators, Complete the Code (K4)	6
Q4	else-if ladder or switch? Complete the Code (K4)	6
Appendix	Repository layout, build commands and reproducibility	-

Questions 5 to 7 are not part of this document.

Method used in every section

Each question follows the order required by the assignment: **Predict, Design, Implement, Measure, Analyse, Evaluate**. Predictions were worked out by hand from the C rules before any program was run. Raw counts (function calls, comparisons, loop iterations) are the primary evidence; the screenshots only confirm the final output. Counts are deterministic and can be reproduced exactly; the few timing figures depend on the machine and are used only as supporting observations.

Q2 - When Does Short-Circuit Evaluation Matter?

Railway senior-citizen concession. 4 marks, Bloom K4 (Analyse).

2.1 Problem analysis and experiment design

Rule used in the experiments: **concession = (senior AND eligible class) OR (disability-card holder AND card verifies)**. The function `verify_card()` stands for the slow server call; every call increments a counter. Passenger data are generated by a fixed pseudo-random generator, so every version sees exactly the same 1000 passengers.

Generated data (target 25% senior, 40% eligible class, 8% card holders). Actual counts: senior = 248, eligible = 405, card holders = 89, senior AND eligible = 99.

Figure 2 is the student's interactive single-passenger program using the rule `(senior && eligible) || disability || verify_card()`. With inputs 1, 1, 0 it printed "Concession Granted" and **Verification calls = 0**: the left side of the `||` was already true, so the slow function was never called.

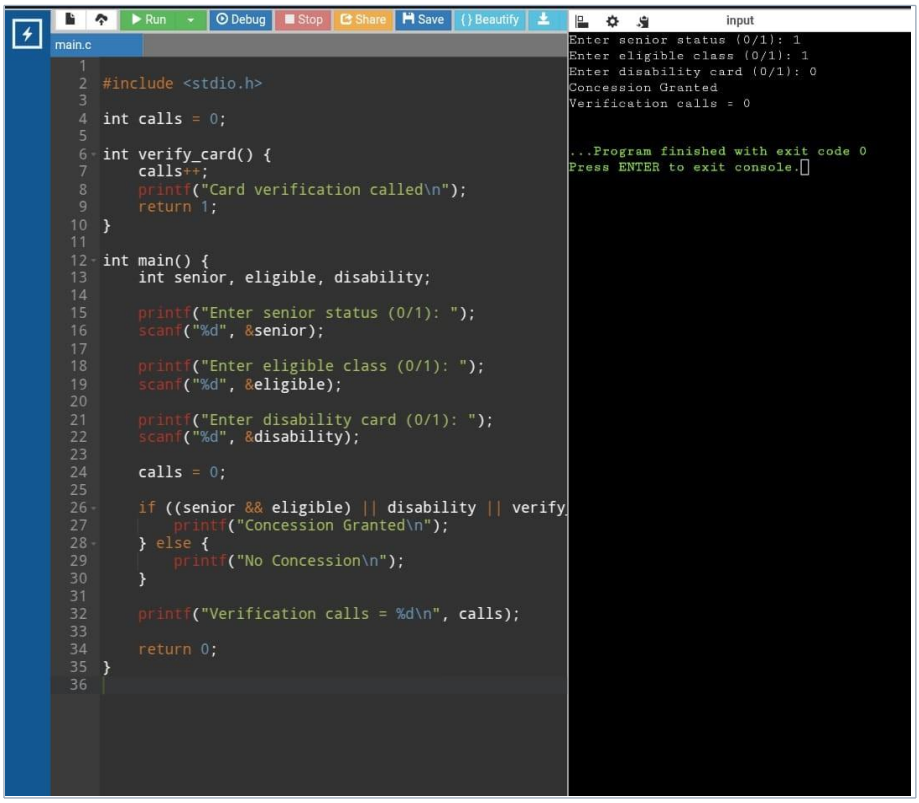


Figure 2 - Student's execution: the slow function is skipped (calls = 0) because (senior && eligible) is already true.

2.2 Experiment 1: when is the second operand not evaluated?

Each side-effect assignment below shows whether the right operand ran (a flag changes to 1 only if it was evaluated).

```
0 && (a=1): result=0 a=0
1 && (b=1): result=1 b=1
1 || (c=1): result=1 c=0
0 || (d=1): result=1 d=1
0 &  (e=1): result=0 e=1
1 |  (f=1): result=1 f=1
2 & 1 = 0    but    2 && 1 = 1
```

Expression	Right operand evaluated?	Reason
0 && (a = 1)	No (a stays 0)	Left is false, so the && result is already 0
1 && (b = 1)	Yes	Left is true, the right operand decides the result
1 || (c = 1)	No (c stays 0)	Left is true, so the || result is already 1
0 || (d = 1)	Yes	Left is false, the right operand decides
0 & (e = 1) and 1 | (f = 1)	Always (e = f = 1)	& and | are bitwise operators: no short-circuit
2 & 1 versus 2 && 1	-	2 & 1 = 0 (bits 10 and 01 share nothing) but 2 && 1 = 1. On conditions that are not exactly 0 or 1, & gives wrong answers.

Difference between & and &&: && is a logical operator that guarantees left-to-right order, stops as soon as the result is known, and treats any non-zero value as true. & is a bitwise operator on the bits of both operands; it always evaluates both and gives no ordering guarantee.

2.3 Experiment 2: order of conditions and the number of slow calls (1000 passengers)

```
V1  (S && E) || (C && verify())     cheap check first, card guard before the slow call
V2  (C && verify()) || (S && E)     card guard first
V3  (verify() && C) || (S && E)     slow call written first
V4  (S & E) | (C & verify())        non-short-circuit operators
V5  nested if (same logic as V1)
V6  (E && S) || (C && verify())     cheap conditions swapped
```

```
V1 (S&&E)||(C&&verify)          slow calls=  78  granted=177  audit entries=  78  senior/elig evals=1000/248 (sum 1248)
V2 (C&&verify)||(S&&E)          slow calls=  89  granted=177  audit entries=  89  senior/elig evals=911/227 (sum 1138)
V3 (verify&&C)||(S&&E)          slow calls=1000  granted=177  audit entries=1000  senior/elig evals=911/227 (sum 1138)
V4 (S&E)|(C&verify) bitwise     slow calls=1000  granted=177  audit entries=1000  senior/elig evals=1000/1000 (sum 2000)
V5 nested if (same as V1)       slow calls=  78  granted=177  audit entries=  78  senior/elig evals=1000/248 (sum 1248)
V6 (E&&S)||(C&&verify)          slow calls=  78  granted=177  audit entries=  78  senior/elig evals=405/1000 (sum 1405)
```

Version	Slow calls	Granted	Cheap evaluations (senior + eligible)	Weighted cost (slow call = 1000 cheap)	Relative to V1
V1	78	177	1248	79,248	1.00
V2	89	177	1138	90,138	1.14
V3	1000	177	1138	1,001,138	12.63
V4	1000	177	2000	1,002,000	12.64
V5	78	177	1248	79,248	1.00
V6	78	177	1405	79,405	1.00

The weight of 1000 cheap evaluations per slow call is an assumption made for illustration; the slow-call counts themselves are measured.

Observations

- All six versions grant the same 177 concessions, so they are logically equivalent. Only the cost differs.
- **V3 and V4 call the slow function 1000 times** (once per passenger): V3 because verify() is written first, V4 because & and | evaluate both sides.
- **V2 calls it 89 times**, exactly the number of card holders, because C guards the slow call.
- **V1 calls it 78 times**: the 89 card holders minus the 11 who already qualified through (senior AND eligible), which is tested first and skips the right side of the ||.
- **V5 (nested if) equals V1 (78 = 78)**: short-circuit operators are just a compact form of the same branching.
- **V1 versus V6:** swapping the cheap conditions does not change the slow calls (78), but cheap evaluations rise from 1248 to 1405. Senior is true for 24.8% of passengers and eligible for 40.5%, so testing senior first stops earlier more often.

2.4 Experiment 3: effect of how many passengers hold a card

```
card%   V1      V2      V3      V4
1       8       8       1000    1000
8       78      89      1000    1000
25      229     251     1000    1000
50      474     517     1000    1000
90      814     893     1000    1000
```

V3 and V4 stay at 1000 whatever the data. V1 and V2 grow with the proportion of card holders and never exceed it. Approximate model: V1 makes n x p_{card} x (1 - $p_{S \text{ and } E}$) calls, V2 makes n x p_{card}, V3 and V4 make n. For 8% card holders: 1000 x 0.089 x 0.901 = 80 (measured 78) and 89 (measured 89).

2.5 When should the cheapest or most-likely-false condition come first?

- **For A && B:** put first the condition that is cheap and likely to be false, because then B is skipped most often. Expected cost = cost(A) + P(A true) x cost(B). Here the card check (false for 91% of passengers, and very cheap) must guard the slow call.
- **For A || B:** put first the condition that is cheap and likely to be *true*, because then B is skipped. This is why (senior && eligible) before the card branch saves 11 slow calls.
- **Order is irrelevant** when both operands are side-effect-free, cost the same and have the same probability, or when both must be evaluated anyway (& and |). It is **never** irrelevant when the first operand guards the second, as in `p != NULL && p->x`.

2.6 Changed condition: the verification function now writes an audit log

In the experiment every call to verify_card() also appends the passenger to an audit log. The audit entries equal the slow calls: V1 = 78, V2 = 89, V3 and V4 = 1000. Re-evaluation:

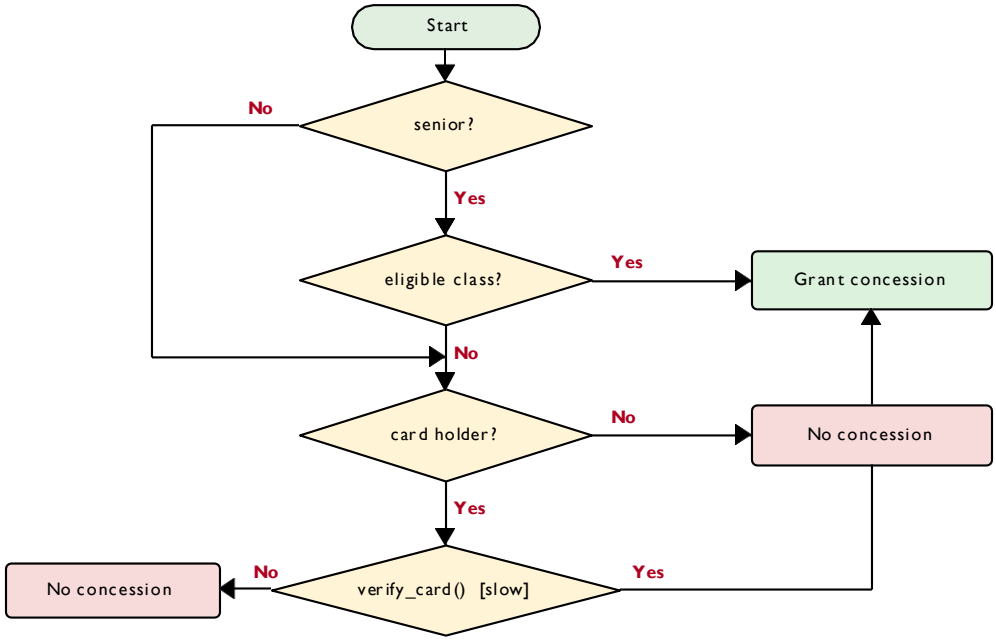
- **V3 and V4 become clearly unacceptable:** they log 911 passengers who hold no card. That wastes storage and one log write per call, and records people who were never verified.
- **V1 versus V2 now depends on the audit rule.** If the log must contain every card holder who presented a card, V1 is wrong because it silently skips the 11 card holders already granted by (senior AND eligible); the log would hold 78 instead of 89 entries. **Recommended order for that policy: V2 (card guard first)**, at the cost of 11 more slow calls (1.1% of passengers).
- If the log only needs the verifications that were actually required for a decision, V1 remains best (fewest calls).
- General rule learned: **do not hide a required side effect in the right operand of && or ||**, because it runs only conditionally. When the log entry is mandatory, call the function explicitly, e.g. `ok = has_card() ? verify_card() : 0;`, then combine the results.

2.7 Alternative concept: nested if, or the non-short-circuit operators

Version	Slow calls for n passengers	Measured (n = 1000)	Time class
&& / || with guard first (V1, V2)	between 0 and n; about n x p_{card} x (1 - $p_{S \text{ and } E}$) for V1, n x p_{card} for V2	78 and 89	O(n) worst case, small constant
Nested if (V5)	same as V1 (the same branches are taken)	78	O(n) worst case, same as V1
Slow call written first, or & and | (V3, V4)	exactly n	1000	Theta(n), always the maximum

Space is O(1) for all versions (only counters), except the audit log, which grows with the number of calls. Nested if gives no speed advantage over a well-ordered && / ||; it is preferable only when each branch needs a different action (for example a different message). The non-short-circuit operators remove the saving completely and should not be used on conditions.

Evaluation flow of V1 / V5 (diagram)



Both No branches of the first two tests join the card check. The slow function is reached only through the card-holder test, which is why V1 and V2 call it so rarely.

Pseudocode

```
FOR each passenger p
    granted <- FALSE
    IF p.senior AND p.eligible THEN granted <- TRUE          (cheap, tested first)
    IF NOT granted AND p.hasCard THEN                          (cheap guard)
        IF verify_card(p) THEN granted <- TRUE                 (slow call, only when needed)
    PRINT granted
```

Source code

```
#include <stdio.h>
#include <stdlib.h>
/* Q2: short-circuit experiments for the railway concession rule.
   Rule: concession = (senior AND eligible class) OR (disability card holder AND card verifies). */
#define N 1000

static unsigned long long seed = 12345;
static int rnd(int mod) { seed = seed * 6364136223846793005ULL + 1442695040888963407ULL; return (int)((seed >> 33) % (unsigned)mod); }

int senior_[N], elig_[N], card_[N];
unsigned long slow_calls, s_calls, e_calls;
int audit_log[N]; int audit_len;
int cur;                                /* index of passenger being processed */

int is_senior(void)   { s_calls++; return senior_[cur]; }
int is_eligible(void) { e_calls++; return elig_[cur]; }
int has_card(void)    { return card_[cur]; }
int verify_card(void) { slow_calls++; audit_log[audit_len++] = cur; return 1; } /* slow server call, card is genuine */

void generate(int pc_senior, int pc_elig, int pc_card)
{
    seed = 12345;
    for (int i = 0; i < N; i++) {
        senior_[i] = rnd(100) < pc_senior;
        elig_[i]   = rnd(100) < pc_elig;
        card_[i]   = rnd(100) < pc_card;
    }
}

typedef int (*Rule)(void);
int v1(void) { return (is_senior() && is_eligible()) || (has_card() && verify_card()); }
int v2(void) { return (has_card() && verify_card()) || (is_senior() && is_eligible()); }
int v3(void) { return (verify_card() && has_card()) || (is_senior() && is_eligible()); }
int v4(void) { return (is_senior() & is_eligible()) | (has_card() & verify_card()); }
int v5(void)
{   /* nested-if equivalent of v1 */
    int g = 0;
    if (is_senior()) { if (is_eligible()) g = 1; }
    if (!g) { if (has_card()) { if (verify_card()) g = 1; } }
    return g;
}
int v6(void) { return (is_eligible() && is_senior()) || (has_card() && verify_card()); } /* cheap order swapped */
```

```
void run(const char *name, Rule r, int show_cheap)
{
    slow_calls = s_calls = e_calls = 0; audit_len = 0;
    int granted = 0;
    for (cur = 0; cur < N; cur++) granted += r();
    printf("%-34s slow calls=%4lu  granted=%3d  audit entries=%4d", name, slow_calls, granted, audit_len);
    if (show_cheap) printf("  senior/elig evals=%lu/%lu (sum %lu)", s_calls, e_calls, s_calls + e_calls);
    printf("\n");
}

int main(void)
{
    printf("== Part 1: 1000 passengers (25%% senior, 40%% eligible class, 8%% card holders) ==\n");
    generate(25, 40, 8);
    int ns = 0, ne = 0, nc = 0, nse = 0;
    for (int i = 0; i < N; i++) { ns += senior_[i]; ne += elig_[i]; nc += card_[i]; nse += senior_[i] && elig_[i]; }
    printf("actual: senior=%d eligible=%d card=%d senior&&eligible=%d\n", ns, ne, nc, nse);
    run("V1 (S&&E)||(C&&verify)",     v1, 1);
    run("V2 (C&&verify)||(S&&E)",     v2, 1);
    run("V3 (verify&&C)||(S&&E)",     v3, 1);
    run("V4 (S&E)|(C&verify) bitwise", v4, 1);
    run("V5 nested if (same as V1)",  v5, 1);
    run("V6 (E&&S)||(C&&verify)",     v6, 1);

    printf("\n== Part 2: card-holder prevalence sweep (slow calls out of 1000) ==\n");
    printf("%-8s %-6s %-6s %-6s %-6s\n", "card%", "V1", "V2", "V3", "V4");
    int sweep[] = {1, 8, 25, 50, 90};
    for (int k = 0; k < 5; k++) {
        generate(25, 40, sweep[k]);
        unsigned long c[4]; Rule rs[4] = {v1, v2, v3, v4};
        for (int j = 0; j < 4; j++) { slow_calls = 0; audit_len = 0; for (cur = 0; cur < N; cur++) rs[j](); c[j] = slow_calls; }
        printf("%-8d %-6lu %-6lu %-6lu %-6lu\n", sweep[k], c[0], c[1], c[2], c[3]);
    }

    printf("\n== Part 3: when is the 2nd operand NOT evaluated? ==\n");
    int a = 0, b = 0, c = 0, d = 0, e = 0, f = 0, g = 0, h = 0;
    int r1 = 0 && (a = 1);   /* && : left false -> right skipped */
    int r2 = 1 && (b = 1);   /* && : left true  -> right evaluated */
    int r3 = 1 || (c = 1);   /* || : left true  -> right skipped */
    int r4 = 0 || (d = 1);   /* || : left false -> right evaluated */
    int r5 = 0 & (e = 1);    /* &  : always evaluates both */
    int r6 = 1 | (f = 1);    /* |  : always evaluates both */
    int r7 = 2 & 1;          /* bitwise: 10 & 01 = 0 although both operands are 'true' */
    int r8 = 2 && 1;         /* logical: 1 */
    (void)g; (void)h;
    printf("0 && (a=1): result=%d a=%d\n", r1, a);
    printf("1 && (b=1): result=%d b=%d\n", r2, b);
    printf("1 || (c=1): result=%d c=%d\n", r3, c);
    printf("0 || (d=1): result=%d d=%d\n", r4, d);
    printf("0 &  (e=1): result=%d e=%d\n", r5, e);
    printf("1 |  (f=1): result=%d f=%d\n", r6, f);
    printf("2 & 1 = %d   but   2 && 1 = %d\n", r7, r8);
    return 0;
}
```